

NetManagement

Sistema de Monitorización y Control
de Equipos en Aulas Informáticas

Memoria Justificativa del Proyecto Final de Ciclo

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Multiplataforma (DAM)

Autor: Enooc Domínguez Quiroga

Centro: IES San Mamede

Curso: 2025 / 2026

Fecha: Junio de 2026

Historial de versiones

Versión	Fecha	Descripción
0.1	Enero 2026	Estructura inicial del proyecto: backend Django con modelos de dispositivos y usuarios, API REST básica con autenticación JWT.
0.2	Febrero 2026	Agente cliente Python funcional con WebSocket, heartbeat y capturas de pantalla. Frontend React con dashboard básico.
0.3	Marzo 2026	Integración con API de MikroTik: bloqueo/desbloqueo de internet por dispositivo y por aula (Network Fencing L2+L3).
0.4	Abril 2026	Sistema RBAC completo, gestión de aulas, equipos de red, hosts permitidos. Tema claro/oscuro en el frontend.
0.5	Mayo 2026	Monitor de heartbeat con Redis keyspace notifications. Auto-descubrimiento de puertos del switch. Instalador del agente como servicio del SO.
0.6	Mayo 2026	Empaquetado del agente como binario autocontenido con PyInstaller. Script de compilación <code>build.py</code> con spec de PyInstaller multiplataforma.
0.7	Mayo 2026	Integración de Tauri 2 en el frontend. La interfaz web se empaqueta también como aplicación de escritorio nativa multiplataforma (Windows, Linux, macOS).
1.0	Junio 2026	Despliegue en producción con Docker Compose y Caddy como proxy inverso con TLS automático. Documentación final completa.

Índice

1	Introducción	6
1.1	Motivación del proyecto	6
1.2	Objetivos del proyecto	6
1.3	Descripción general del sistema	7
2	Definición del alcance	7
2.1	Qué hace el proyecto	7
2.2	Qué NO hace el proyecto	8
2.3	Interacción con sistemas externos	8
2.4	Definición del MVP (Producto Mínimo Viable)	9
3	Análisis de requisitos	10
3.1	Requisitos funcionales	10
3.2	Requisitos no funcionales	10
4	Casos de uso	11
4.1	Actores del sistema	11
4.2	Diagrama de casos de uso	12
4.3	Descripción de los casos de uso principales	12
4.3.1	CU-01: Ver dashboard en tiempo real	12
4.3.2	CU-02: Bloquear internet de un dispositivo	13
4.3.3	CU-03: Bloquear internet de un aula completa	13
4.3.4	CU-04: Apagar equipo remoto	14
5	Arquitectura del sistema	14
5.1	Visión general de la arquitectura	14
5.2	Diagrama de despliegue	15
5.3	Componentes del sistema	15
5.3.1	Backend (Django 5 + DRF + Channels)	15
5.3.2	Frontend (React 19 + Vite + Material UI 7 + Tauri 2)	16
5.3.3	Agente cliente (Python)	16
5.3.4	Proxy inverso (Caddy 2)	16
5.4	Redes Docker	17
6	Diseño de la base de datos	17
6.1	Descripción de las entidades	22
7	Diseño técnico y justificación de tecnologías	22
7.1	Stack tecnológico	22
7.2	API REST — Endpoints principales	24
7.2.1	Autenticación	24
7.2.2	Usuarios y grupos	25
7.2.3	Dispositivos	25
7.2.4	Aulas	25
7.2.5	Equipos de red y hosts permitidos	26
7.3	Protocolo WebSocket	26
7.3.1	Mensajes del Agente → Servidor	26

7.3.2	Mensajes del Servidor → Agente	27
7.3.3	Mensajes del Servidor → Dashboard	27
7.4	Flujo de datos detallado	27
7.5	Seguridad	28
7.5.1	Autenticación y autorización	28
7.5.2	Cifrado y almacenamiento seguro	28
7.5.3	CORS y CSRF	28
8	Estructura del proyecto	29
9	Implementación	30
9.1	Backend — Aspectos destacados	30
9.1.1	Cifrado transparente de contraseñas de red	30
9.1.2	Monitor de heartbeat con Redis keypace	31
9.1.3	Servicio MikroTik — Network Fencing	31
9.2	Frontend — Aspectos destacados	31
9.2.1	Gestión de tokens JWT con interceptores	32
9.2.2	Capturas de pantalla con Nanostores	32
9.2.3	Rutas protegidas y permisos	32
9.3	Agente cliente — Aspectos destacados	32
9.3.1	Compilación como binario autocontenido (PyInstaller)	32
9.3.2	Instalación como servicio del sistema	33
9.3.3	Reconexión automática	33
9.4	Aplicación de escritorio — Tauri 2	33
9.4.1	Estructura del proyecto Tauri	33
9.4.2	Configuración de la ventana	34
9.4.3	Flujo de compilación	34
10	Despliegue	34
10.1	Requisitos del entorno	35
10.2	Servicios Docker	35
10.3	Proceso de despliegue	35
10.4	Variables de entorno	36
11	Pruebas	36
11.1	Pruebas funcionales realizadas	36
11.2	Entornos de prueba	37
12	Complicaciones encontradas	37
12.1	Instalación del agente como servicio en Windows	37
12.2	Capturas de pantalla en Linux con Wayland	38
12.3	Certificados TLS con Caddy y conexiones WSS del agente	38
12.4	Conexión con la API de MikroTik	39
13	Conclusiones	39
13.1	Objetivos alcanzados	39
13.2	Lecciones aprendidas	40
14	Líneas de trabajo futuro	40

15 Bibliografía y referencias	41
--	-----------

1 Introducción

1.1 Motivación del proyecto

La gestión de aulas informáticas en centros educativos supone un reto recurrente para los docentes y administradores de sistemas. Las tareas cotidianas —como comprobar qué equipos están encendidos, enviar órdenes de apagado remoto, o restringir el acceso a internet durante un examen— requieren herramientas específicas que, en muchos casos, no están disponibles o resultan costosas y difíciles de integrar con la infraestructura existente.

Las soluciones comerciales de gestión de aula (por ejemplo, *Veyon*, *LanSchool* o *NetSupport School*) proporcionan funcionalidades de monitorización de pantallas y control básico, pero carecen de una característica crítica para entornos con evaluaciones presenciales: el **control de red a nivel físico**. Es decir, la capacidad de bloquear o aislar el tráfico de red de un equipo actuando directamente sobre el puerto del switch al que está conectado.

Este proyecto nace de la necesidad real de un centro educativo que dispone de infraestructura de red **MikroTik** y busca una solución que integre:

- Monitorización en tiempo real del estado de los equipos de un aula.
- Control remoto de los equipos (apagado, reinicio).
- Visualización de capturas de pantalla de los equipos de los alumnos.
- **Aislamiento dinámico de red** (“Network Fencing”) mediante la API de MikroTik, actuando sobre reglas de firewall y *split-horizon* de los puertos del bridge.

1.2 Objetivos del proyecto

Los objetivos principales de **NetManagement** son:

1. **Centralizar la gestión de aulas informáticas** en una única interfaz web accesible desde cualquier navegador.
2. **Proporcionar monitorización en tiempo real** del estado de los equipos mediante WebSockets, incluyendo detección automática de encendido/apagado y visualización de capturas de pantalla.
3. **Permitir el control remoto** de los equipos del aula: apagado, reinicio y otras acciones.
4. **Implementar el aislamiento de red dinámico** (*Network Fencing*) interactuando con equipos de red MikroTik, permitiendo bloquear internet a nivel individual o por aula completa.
5. **Gestionar usuarios con control de acceso basado en roles** (RBAC), diferenciando entre administradores y profesores con aulas asignadas.
6. **Facilitar el despliegue** mediante contenedores Docker y un proxy inverso con TLS automático.

1.3 Descripción general del sistema

NetManagement es un sistema distribuido compuesto por cuatro capas:

1. **Capa de gestión (Frontend):** Aplicación web SPA desarrollada con React 19, Material UI 7 y Vite. Permite a los profesores y administradores visualizar el estado de los equipos, enviar comandos y gestionar la configuración del sistema.
2. **Servidor central (Backend):** API REST y servidor de WebSockets basado en Django 5, Django REST Framework y Django Channels (Daphne). Recibe las peticiones del frontend, gestiona la lógica de negocio, y se comunica tanto con los agentes como con los equipos de red MikroTik.
3. **Capa de infraestructura de red:** Los routers y switches MikroTik ejecutan las reglas de firewall y aislamiento de red dictadas por el backend a través de su API nativa (librouters).
4. **Capa de aula (Agentes):** Un cliente ligero en Python se instala en cada equipo del aula. Se conecta al servidor por WebSocket, reporta información del sistema, envía capturas de pantalla periódicas y ejecuta los comandos recibidos (apagado, reinicio, etc.).
5. **Capa de escritorio (Tauri):** Opcionalmente, el frontend React puede distribuirse también como aplicación de escritorio nativa mediante Tauri 2, sin necesidad de instalar un navegador independiente. El núcleo de la aplicación en Rust actúa como contenedor de la SPA, gestionando la ventana nativa del sistema operativo.

2 Definición del alcance

2.1 Qué hace el proyecto

NetManagement cubre las siguientes funcionalidades:

- **Gestión de aulas:** Creación, edición y eliminación de aulas. Asignación de dispositivos a aulas.
- **Gestión de dispositivos monitorizados:** Registro automático de equipos al conectarse el agente. Visualización del estado (online/offline), IP, hostname, MAC y puerto del switch.
- **Monitorización en tiempo real:** Actualización instantánea del estado de los equipos vía WebSocket. Capturas de pantalla periódicas retransmitidas al dashboard sin almacenamiento persistente.
- **Control remoto de equipos:** Apagado remoto de equipos individuales.
- **Network Fencing — Bloqueo de internet:**
 - Bloqueo individual por dispositivo (reglas IP firewall + split-horizon en el puerto del switch).
 - Bloqueo por aula completa (aplica reglas a todos los dispositivos del aula).

- Hosts permitidos configurables (IPs que permanecen accesibles durante el bloqueo, por ejemplo, el servidor de exámenes).
- **Gestión de equipos de red:** CRUD de switches/routers MikroTik con credenciales cifradas (Fernet).
- **Gestión de usuarios y permisos:**
 - Autenticación JWT con refresco automático de tokens.
 - Modelo RBAC con grupos y permisos granulares de Django.
 - Administradores: acceso completo a toda la configuración.
 - Profesores: acceso limitado a las aulas que tienen asignadas.
- **Detección automática de equipos offline:** Sistema de heartbeat basado en claves TTL de Redis con notificaciones *keyspace*.
- **Auto-descubrimiento de puertos de red:** Al conectarse un agente, el backend consulta la tabla L2 del bridge MikroTik para mapear la MAC al puerto físico del switch.

2.2 Qué NO hace el proyecto

Es importante delimitar claramente las funciones que quedan fuera del alcance de este sistema:

- **No es un software de control de aula tradicional:** No permite ver la pantalla de los alumnos en tiempo real con vídeo en streaming de alta calidad. Las capturas de pantalla son imágenes estáticas periódicas (JPEG comprimido).
- **No gestiona la configuración de los equipos de red MikroTik:** El sistema solo crea y elimina reglas de firewall y modifica el split-horizon de puertos específicos. No configura VLANs, enrutamiento ni otros aspectos de la infraestructura de red.
- **No incluye inventario completo de hardware/software:** Recoge información básica del sistema (SO, CPU, RAM, disco), pero no sustituye a herramientas de inventario como GLPI o OCS Inventory.
- **No implementa Wake-on-LAN:** El encendido remoto de equipos queda fuera del MVP.
- **No tiene chat ni mensajería integrada:** No se puede enviar mensajes a los equipos del aula.
- **No es multiplataforma en el frontend:** La interfaz de administración es exclusivamente web. No existe una aplicación móvil nativa.

2.3 Interacción con sistemas externos

El único sistema externo con el que interactúa NetManagement es la **infraestructura de red MikroTik**. Esta interacción se realiza de la siguiente manera:

- El backend se conecta a la API de MikroTik RouterOS mediante la librería Python `librouteros`.
- La conexión se realiza por el puerto API estándar (8728) o por el puerto API-SSL (8729) con TLS.
- Las operaciones se limitan a:
 - Consulta de la tabla `/interface/bridge/host` para auto-descubrir puertos.
 - Consulta de la tabla `/ip/arp` como alternativa.
 - Creación y eliminación de reglas en `/ip/firewall/filter`.
 - Modificación del `horizon` en `/interface/bridge/port`.
 - Activación/desactivación de `use-ip-firewall` en `/interface/bridge/settings`.
- Los equipos de red MikroTik **no forman parte del proyecto**; son infraestructura preexistente del centro. El proyecto asume que están correctamente configurados con un bridge y acceso API habilitado.

2.4 Definición del MVP (Producto Mínimo Viable)

Las funcionalidades que conforman el MVP son:

1. Registro automático y monitorización de equipos (online/offline) en tiempo real.
2. Dashboard web con visualización por aulas y capturas de pantalla periódicas.
3. Apagado remoto de equipos individuales.
4. Bloqueo/desbloqueo de internet por dispositivo y por aula (Network Fencing vía MikroTik).
5. Gestión de aulas, dispositivos de red, hosts permitidos y usuarios.
6. Autenticación JWT y sistema de permisos por roles.
7. Despliegue en producción con Docker Compose y Caddy.

Funcionalidades fuera del MVP (ampliaciones futuras):

- Wake-on-LAN para encendido remoto.
- Streaming de pantalla en vídeo en tiempo real.
- Envío de mensajes y notificaciones a los equipos.
- Chat entre profesor y alumno.
- Transferencia de archivos al equipo del alumno.
- Registro de historial de actividad y auditoría.
- Aplicación móvil nativa para profesores.
- Integración con sistemas de directorio (LDAP/Active Directory).

3 Análisis de requisitos

3.1 Requisitos funcionales

ID	Descripción
RF-01	El sistema debe permitir el registro automático de equipos al conectarse el agente.
RF-02	El sistema debe mostrar el estado en tiempo real de cada equipo (online/offline).
RF-03	El sistema debe permitir la visualización de capturas de pantalla periódicas.
RF-04	El sistema debe permitir el apagado remoto de equipos individuales.
RF-05	El sistema debe permitir bloquear/desbloquear el acceso a internet de un equipo individual actuando sobre el equipo de red MikroTik.
RF-06	El sistema debe permitir bloquear/desbloquear el acceso a internet de todos los equipos de un aula simultáneamente.
RF-07	El sistema debe permitir configurar hosts permitidos (IPs accesibles durante el bloqueo).
RF-08	El sistema debe gestionar aulas: crear, editar, eliminar y asignar dispositivos.
RF-09	El sistema debe gestionar equipos de red (switches/routers MikroTik): crear, editar y eliminar, almacenando credenciales de forma cifrada.
RF-10	El sistema debe implementar autenticación basada en tokens JWT con refresco automático.
RF-11	El sistema debe implementar RBAC con permisos granulares basados en grupos de Django.
RF-12	Los administradores deben poder gestionar usuarios, grupos y permisos.
RF-13	Los profesores solo deben poder ver y actuar sobre las aulas que tienen asignadas.
RF-14	El sistema debe auto-descubrir el puerto físico del switch al que está conectado cada equipo, consultando la tabla L2 del bridge MikroTik.
RF-15	El sistema debe detectar automáticamente cuando un equipo deja de responder (heartbeat expirado) y marcarlo como offline.
RF-16	El sistema debe proporcionar un agente multiplataforma (Windows y Linux) que se instale como servicio del sistema.

3.2 Requisitos no funcionales

ID	Descripción
RNF-01	La interfaz web debe ser responsiva y compatible con navegadores modernos (Chrome, Firefox, Edge).
RNF-02	La comunicación en tiempo real debe tener una latencia inferior a 2 segundos en red local.
RNF-03	Las credenciales de los equipos de red deben almacenarse cifradas con Fernet (AES-128-CBC).
RNF-04	El sistema debe soportar al menos 50 equipos conectados simultáneamente.
RNF-05	El despliegue debe realizarse con contenedores Docker para facilitar la instalación.
RNF-06	El proxy inverso debe gestionar el TLS de forma automática (certificados Let's Encrypt vía Caddy).
RNF-07	La aplicación debe seguir los principios REST en el diseño de la API.
RNF-08	Las contraseñas de usuario deben almacenarse con hash seguro (PBKDF2 de Django).
RNF-09	El agente cliente debe funcionar en Windows 10+ y distribuciones Linux con escritorio.
RNF-10	El sistema debe ser escalable horizontalmente añadiendo más instancias del backend detrás del proxy.
RNF-11	El frontend debe poder distribuirse como aplicación de escritorio nativa (Windows, Linux, macOS) mediante Tauri 2, además de como SPA web.
RNF-12	El agente cliente debe poder compilarse como un binario autocontenido (sin dependencias externas) usando PyInstaller, para facilitar su distribución masiva en los equipos del aula.

4 Casos de uso

4.1 Actores del sistema

- **Administrador:** Usuario con acceso completo. Puede gestionar toda la configuración del sistema: usuarios, grupos, permisos, aulas, equipos de red, hosts permitidos y dispositivos.
- **Profesor:** Usuario con permisos limitados. Solo puede visualizar y actuar sobre las aulas que tiene asignadas: ver el estado de los equipos, apagar equipos, bloquear/desbloquear internet, y ver capturas de pantalla.
- **Agente (sistema):** Programa instalado en cada equipo del aula. Se conecta automáticamente al servidor, envía información del sistema, capturas de pantalla y ejecuta los comandos recibidos.
- **Equipo de red MikroTik (sistema externo):** Infraestructura de switching a la que el backend se conecta para aplicar reglas de aislamiento de red.

4.2 Diagrama de casos de uso

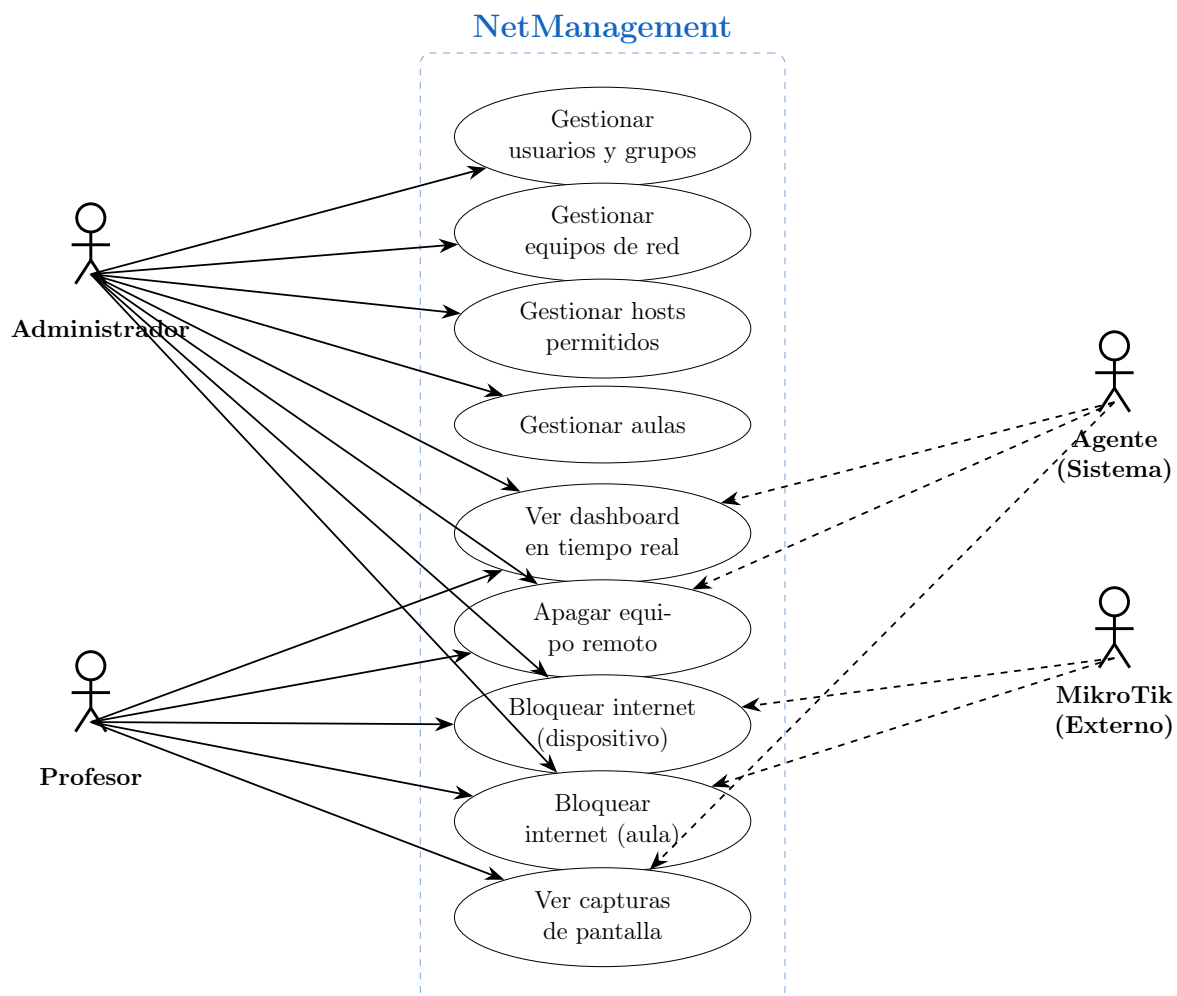


Figura 1: Diagrama de casos de uso del sistema NetManagement.

4.3 Descripción de los casos de uso principales

4.3.1 CU-01: Ver dashboard en tiempo real

Actor principal	Profesor / Administrador
Precondiciones	El usuario está autenticado y tiene permisos de visualización.
Flujo principal	1. El usuario accede al dashboard. 2. El sistema carga la lista de aulas (todas si es admin; solo las asignadas si es profesor). 3. El usuario selecciona un aula. 4. El sistema muestra los dispositivos del aula en una cuadrícula con su estado (online/offline), hostname, IP y la última captura de pantalla. 5. El sistema actualiza el estado automáticamente vía WebSocket.
Postcondiciones	El usuario visualiza el estado actualizado de los equipos.

4.3.2 CU-02: Bloquear internet de un dispositivo

Actor principal	Profesor / Administrador
Precondiciones	El dispositivo tiene un equipo de red asignado y un puerto detectado.
Flujo principal	1. El usuario pulsa el botón de bloqueo de internet en la tarjeta del dispositivo. 2. El backend se conecta a la API del router MikroTik asignado. 3. El backend crea reglas accept para cada host permitido y reglas drop para el resto del tráfico. 4. El backend aplica <i>split-horizon</i> en el puerto del bridge correspondiente. 5. El sistema notifica al dashboard del cambio de estado vía WebSocket.
Flujo alternativo	Si el equipo de red no responde, se devuelve un error 502 (Bad Gateway).
Postcondiciones	El dispositivo pierde acceso a internet excepto a los hosts permitidos.

4.3.3 CU-03: Bloquear internet de un aula completa

Actor principal	Profesor / Administrador
Precondiciones	El aula tiene dispositivos con equipos de red asignados.
Flujo principal	1. El usuario pulsa el botón de bloqueo global en la barra del aula. 2. El backend agrupa los dispositivos del aula por equipo de red. 3. Para cada equipo de red, abre una conexión API y crea las reglas de firewall y split-horizon para todos los dispositivos del aula. 4. Se actualiza el estado de bloqueo del aula y de cada dispositivo en la base de datos. 5. El dashboard recibe la notificación vía WebSocket.
Postcondiciones	Todos los equipos del aula pierden acceso a internet excepto a los hosts permitidos.

4.3.4 CU-04: Apagar equipo remoto

Actor principal	Profesor / Administrador
Precondiciones	El dispositivo está online.
Flujo principal	1. El usuario pulsa el botón de apagado en la tarjeta del dispositivo. 2. El backend envía un mensaje al grupo de canales del agente (identificado por su MAC). 3. El agente recibe el comando <code>shutdown</code> y ejecuta el apagado del sistema operativo. 4. El agente envía un aviso de apagado al servidor antes de cerrar la conexión. 5. El dispositivo se marca como offline en el dashboard.
Flujo alternativo	Si el dispositivo no está online, se devuelve un error 409 (Conflict).
Postcondiciones	El equipo físico se apaga.

5 Arquitectura del sistema

5.1 Visión general de la arquitectura

NetManagement sigue una arquitectura distribuida orientada a eventos con cuatro capas bien diferenciadas. La comunicación entre el frontend y el backend combina peticiones REST para operaciones CRUD con WebSockets para la transmisión en tiempo real.

5.2 Diagrama de despliegue

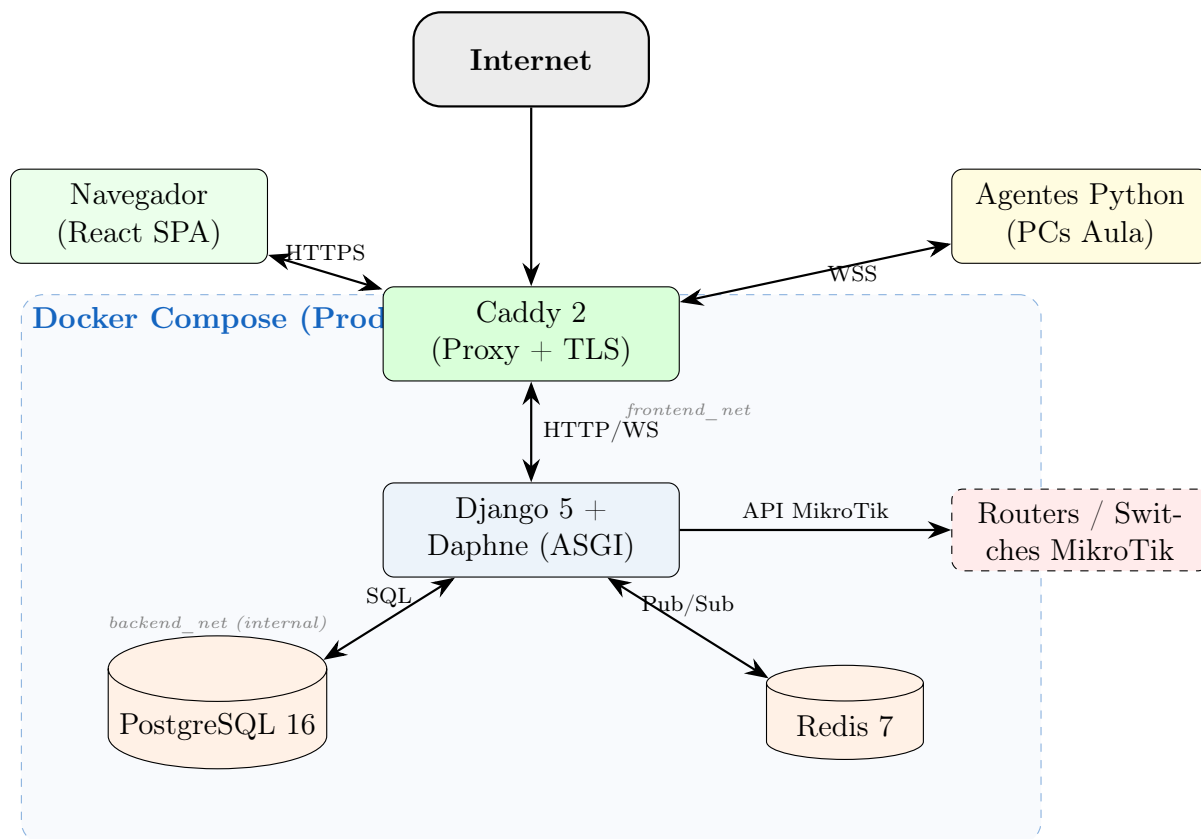


Figura 2: Diagrama de despliegue del sistema NetManagement.

5.3 Componentes del sistema

5.3.1 Backend (Django 5 + DRF + Channels)

El backend es el núcleo del sistema. Está desarrollado en Python con el framework Django 5 y expone:

- Una **API REST** mediante Django REST Framework para todas las operaciones CRUD.
- Un servidor **WebSocket** mediante Django Channels y Daphne (ASGI) para comunicación bidireccional en tiempo real.
- Un módulo de **servicio MikroTik** (`mikrotik_service.py`) que encapsula toda la lógica de interacción con la API de los equipos de red.
- Un **monitor de heartbeat** basado en claves TTL de Redis con notificaciones *keyspace* para detección de equipos offline.

El backend se ejecuta con **Daphne** como servidor ASGI, que maneja tanto las peticiones HTTP como las conexiones WebSocket de forma asíncrona.

5.3.2 Frontend (React 19 + Vite + Material UI 7 + Tauri 2)

La interfaz de usuario es una Single Page Application (SPA) desarrollada con:

- **React 19:** Biblioteca de UI con hooks y componentes funcionales.
- **Vite (rolldown-vite):** Bundler de nueva generación con HMR instantáneo. Se utiliza la variante `rolldown-vite` (v7.2.5) que incorpora el nuevo motor Rolldown escrito en Rust para tiempos de compilación aún más rápidos.
- **Material UI 7:** Biblioteca de componentes con soporte para tema claro/oscuro.
- **Nanostores:** Gestión de estado ligera con persistencia en `sessionStorage`.
- **react-use-websocket:** Hook declarativo para la conexión WebSocket del dashboard.
- **Axios:** Cliente HTTP con interceptores para gestión automática de tokens JWT.
- **Tauri 2:** Framework para empaquetar aplicaciones web como aplicaciones de escritorio nativas. Permite distribuir el frontend como un ejecutable nativo de Windows, Linux o macOS, aprovechando el WebView del sistema operativo y añadiendo un núcleo Rust ligero.

5.3.3 Agente cliente (Python)

Un programa ligero en Python que se instala en cada equipo del aula:

- Se conecta al backend por WebSocket (`wss://`).
- Envía un reporte completo del sistema al conectarse (*startup*).
- Envía heartbeats periódicos con MAC e IP.
- Envía capturas de pantalla periódicas (JPEG comprimido en base64).
- Escucha y ejecuta comandos del servidor (apagado, reinicio, etc.).
- Se instala como servicio del sistema (tarea programada en Windows, servicio systemd en Linux).
- Reconexión automática con backoff exponencial.

5.3.4 Proxy inverso (Caddy 2)

Caddy actúa como punto de entrada único al sistema:

- `/api/*` → Backend Django REST (eliminando el prefijo `/api`).
- `/ws/*` → Backend Daphne (WebSockets).
- `/admin/*` → Panel de administración de Django.
- `/static/*` → Archivos estáticos recopilados con `collectstatic`.

- /* → SPA de React (con `try_files` para soportar el enrutamiento del lado del cliente).
- TLS automático con certificados Let's Encrypt.
- Compresión gzip y zstd.
- Cabeceras de seguridad (X-Content-Type-Options, X-Frame-Options, Referrer-Policy).

5.4 Redes Docker

El despliegue en producción utiliza dos redes Docker aisladas:

Red	Tipo	Propósito
backend_net	bridge (no internal)	Conecta backend, PostgreSQL y Redis. No es <i>internal</i> porque el backend necesita alcanzar los routers MikroTik en la red local.
frontend_net	bridge	Conecta Caddy con el backend. Caddy es el único servicio que expone puertos al exterior (80 y 443).

Cuadro 3: Redes Docker del despliegue en producción.

6 Diseño de la base de datos

A continuación se presentan los diagramas del modelo de datos del sistema. El primero sigue la **notación de Chen** (modelo conceptual con entidades, atributos y relaciones), y el segundo la **notación crow's foot** (modelo relacional más cercano a la implementación física).

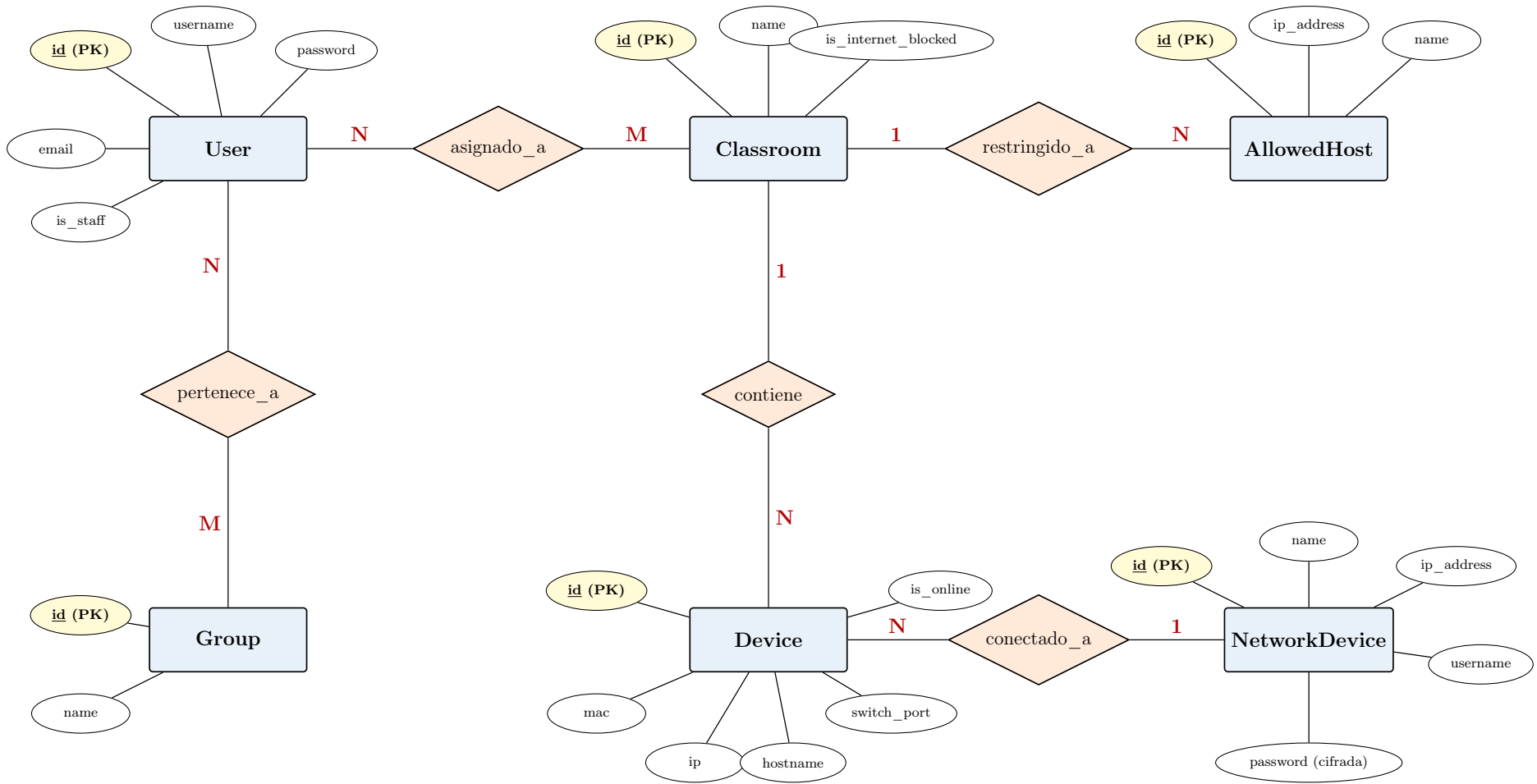
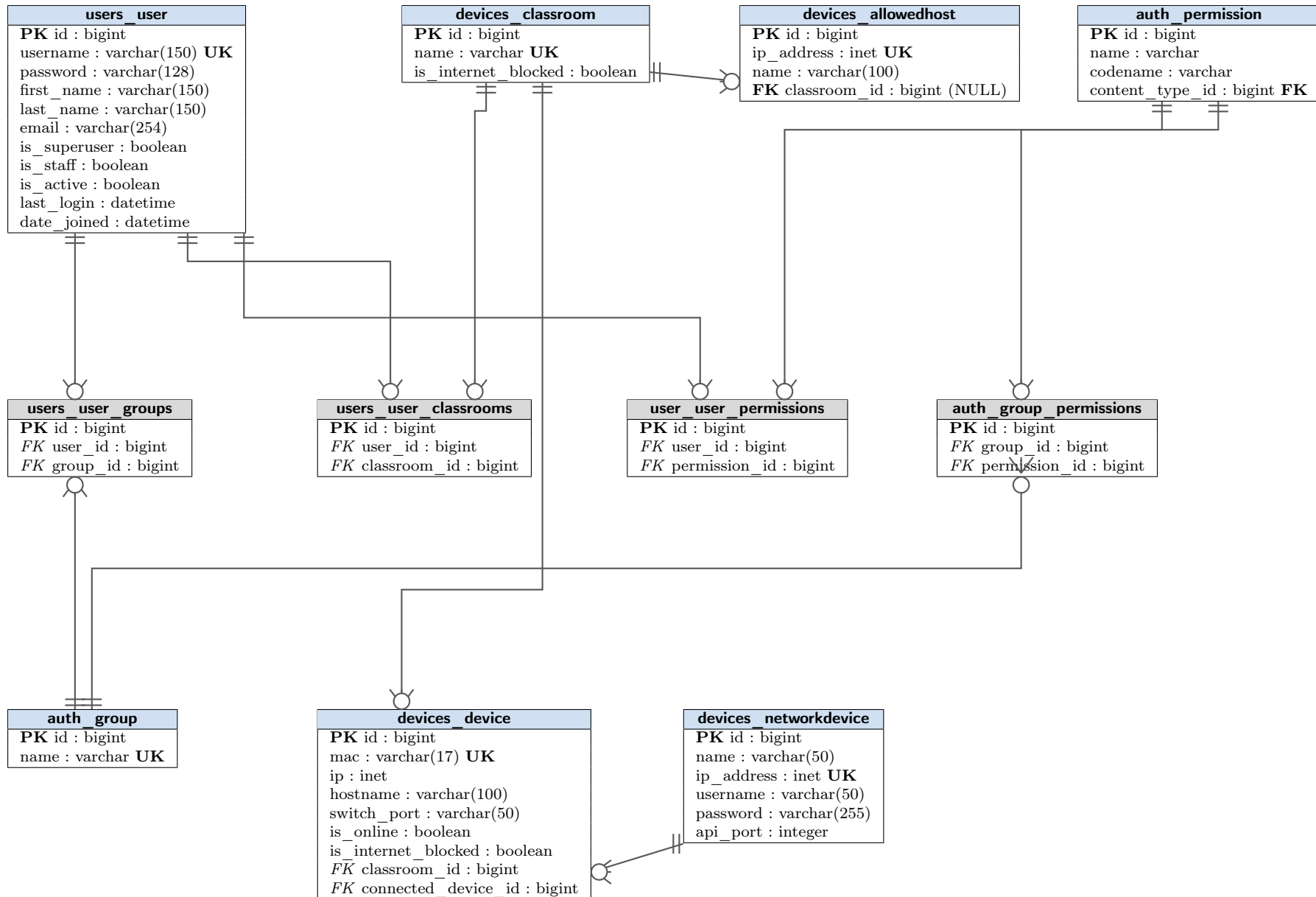


Figura 3: Diagrama Entidad-Relación en notación de Chen.



El modelo relacional se presenta a continuación en formato tabular detallado. Las claves primarias se indican con **PK**, las claves foráneas con *FK* y los campos únicos con **UK**.

Cuadro 4: Tabla `users_user`

Campo	Tipo	Restricciones
id	BIGINT	PK
username	VARCHAR(150)	UNIQUE, NOT NULL
password	VARCHAR(128)	NOT NULL
first_name	VARCHAR(150)	
last_name	VARCHAR(150)	
email	VARCHAR(254)	
is_superuser	BOOLEAN	DEFAULT FALSE
is_staff	BOOLEAN	DEFAULT FALSE
is_active	BOOLEAN	DEFAULT TRUE
last_login	DATETIME	NULL
date_joined	DATETIME	NOT NULL

Cuadro 5: Tabla `devices_classroom`

Campo	Tipo	Restricciones
id	BIGINT	PK
name	VARCHAR	UNIQUE, NOT NULL
is_internet_blocked	BOOLEAN	DEFAULT FALSE

Cuadro 6: Tabla `devices_device`

Campo	Tipo	Restricciones
id	BIGINT	PK
mac	VARCHAR(17)	UNIQUE, NOT NULL
ip	INET	NOT NULL
hostname	VARCHAR(100)	NOT NULL
switch_port	VARCHAR(50)	DEFAULT ''
is_online	BOOLEAN	DEFAULT FALSE
is_internet_blocked	BOOLEAN	DEFAULT FALSE
classroom_id	BIGINT	FK → devices_classroom.id, NULL
connected_device_id	BIGINT	FK → devices_networkdevice.id, NULL

Cuadro 7: Tabla `devices_networkdevice`

Campo	Tipo	Restricciones
id	BIGINT	PK
name	VARCHAR(50)	NOT NULL
ip_address	INET	UNIQUE, NOT NULL
username	VARCHAR(50)	NOT NULL
password	VARCHAR(255)	NOT NULL (cifrada con Fernet)
api_port	INTEGER	DEFAULT 8728

Cuadro 8: Tabla `devices_allowedhost`

Campo	Tipo	Restricciones
id	BIGINT	PK
ip_address	INET	UNIQUE, NOT NULL
name	VARCHAR(100)	BLANK
classroom_id	BIGINT	FK → <code>devices_classroom.id</code> , NULL (opcional)

Cuadro 9: Tabla `auth_group`

Campo	Tipo	Restricciones
id	BIGINT	PK
name	VARCHAR	UNIQUE, NOT NULL

Cuadro 10: Tablas intermedias (relaciones N:M)

Tabla	FK 1	FK 2	Descripción
<code>users_user_classrooms</code>	<code>user_id</code>	<code>classroom_id</code>	Aulas asignadas a cada usuario
<code>users_user_groups</code>	<code>user_id</code>	<code>group_id</code>	Grupos de permisos del usuario
<code>users_user_user_permissions</code>	<code>user_id</code>	<code>permission_id</code>	Permisos individuales
<code>auth_group_permissions</code>	<code>group_id</code>	<code>permission_id</code>	Permisos asignados a cada grupo

Relaciones principales:

- `devices_classroom` ↔ `devices_device`: 1:N (un aula contiene muchos dispositivos).
- `devices_classroom` ↔ `devices_allowedhost`: 1:N (un aula puede tener hosts permitidos específicos; si el host no tiene aula asignada, es global).
- `devices_networkdevice` ↔ `devices_device`: 1:N (un equipo de red conecta muchos dispositivos).

- `users_user` $\longleftrightarrow$ `devices_classroom`: N:M (un usuario puede tener varias aulas asignadas).
- `users_user` $\longleftrightarrow$ `auth_group`: N:M (un usuario puede pertenecer a varios grupos).

6.1 Descripción de las entidades

Entidad	Descripción
<code>users_user</code>	Modelo de usuario personalizado que extiende <code>AbstractUser</code> de Django. Añade una relación muchos-a-muchos con las aulas (<code>classrooms</code>), permitiendo asignar a cada profesor las aulas que puede gestionar.
<code>devices_classroom</code>	Representa un aula del centro. Agrupa dispositivos y almacena el estado global de bloqueo de internet.
<code>devices_device</code>	Dispositivo individual (equipo de un alumno) monitorizado por el sistema. Se identifica por su dirección MAC (única). Registra el equipo de red al que está conectado, el puerto del switch, y los estados de conexión y bloqueo.
<code>devices_networkdevice</code>	Equipo de red MikroTik (switch o router) al que el backend se conecta por API para aplicar reglas de aislamiento. La contraseña se almacena cifrada con Fernet (AES).
<code>devices_allowedhost</code>	IP de un host que siempre será accesible aunque se bloquee internet. Cuando se crea una regla de bloqueo, se añaden reglas <code>accept</code> para cada host permitido antes de la regla <code>drop</code> . Si tiene un aula asignada (<code>classroom_id</code>), solo se aplica en las reglas de bloqueo de esa aula; si no tiene aula (valor nulo), se aplica de forma global en cualquier regla de bloqueo.
<code>auth_group</code>	Grupos de permisos de Django. Permite definir roles (por ejemplo, “Profesor”) con un conjunto específico de permisos.
<code>users_user_classrooms</code>	Tabla intermedia de la relación N:M entre usuarios y aulas.
<code>users_user_groups</code>	Tabla intermedia de la relación N:M entre usuarios y grupos.

7 Diseño técnico y justificación de tecnologías

7.1 Stack tecnológico

Capa	Tecnología	Justificación
Backend	Python 3.11, Django 5, DRF	Django ofrece un ORM robusto, sistema de autenticación y permisos integrado, y una comunidad amplia. DRF proporciona serialización, validación y vistas genéricas para APIs REST con mínima configuración.
Tiempo real	Django Channels + Daphne	Channels extiende Django para soportar WebSockets mediante el protocolo ASGI. Daphne es el servidor ASGI oficial del proyecto Django. Permite manejar HTTP y WebSockets en el mismo proceso.
Frontend	React 19 + Vite + MUI 7	React es la biblioteca de UI más extendida del ecosistema. Vite ofrece tiempos de recarga instantáneos. Material UI proporciona componentes de alta calidad con soporte de temas claro/oscuro.
Estado (frontend)	Nanostores	Librería de gestión de estado ultraligera (<1 KB). Permite persistir datos en <code>sessionStorage</code> de forma declarativa. Se usa para cachear las capturas de pantalla sin llamadas adicionales al servidor.
Base de datos	PostgreSQL 16	Base de datos relacional con alta concurrencia transaccional, fiabilidad probada y rendimiento excelente para cargas mixtas de lectura/escritura.
Caché / Bus	Redis 7	Se usa como broker de Django Channel Layers (pub/sub para WebSockets) y como almacén de claves TTL para el sistema de heartbeat.
Redes	librouteros (Python)	Biblioteca oficial para comunicarse con la API de MikroTik RouterOS desde Python. Soporta conexiones planas y SSL.
Agente	Python (websockets, mss, psutil)	Python permite escribir un cliente ligero y multiplataforma. <code>mss</code> captura pantallas de forma eficiente. <code>psutil</code> recoge información del sistema.

Capa	Tecnología	Justificación
Proxy inverso	Caddy 2	TLS automático con Let's Encrypt sin configuración manual. Soporta HTTP/3, proxy de WebSockets y servicio de ficheros estáticos. Configuración declarativa mínima.
Infraestructura	Docker Compose	Permite definir todos los servicios (backend, BD, Redis, Caddy) en un solo fichero y desplegarlos con un único comando. Garantiza entornos reproducibles.
Cifrado	cryptography (Fernet)	Cifrado simétrico AES-128-CBC para almacenar las contraseñas de los equipos de red en la base de datos. La clave se deriva de la SECRET_KEY de Django.
Escritorio	Tauri 2 (Rust)	Framework para empaquetar la SPA de React como aplicación de escritorio nativa. Utiliza el WebView del sistema operativo (WebView2 en Windows, WebKitGTK en Linux, WKWebView en macOS) con un núcleo Rust mínimo. Es la alternativa moderna a Electron, con un tamaño de binario mucho menor.
Empaquetado agente	PyInstaller	Herramienta para compilar el agente Python y todos sus scripts auxiliares en un único binario autocontenido. Permite distribuir el agente sin necesidad de instalar Python ni dependencias en los equipos del aula.

7.2 API REST — Endpoints principales

La API sigue las convenciones REST. Los endpoints están organizados por recurso y utilizan los ViewSets de DRF con enrutamiento automático.

7.2.1 Autenticación

Método	Endpoint	Descripción
--------	----------	-------------

POST	/api/token/	Obtener par de tokens JWT (access + refresh). Incluye <code>is_staff</code> , <code>is_superuser</code> , permisos y aulas asignadas en la respuesta.
POST	/api/token/refresh/	Refrescar el token de acceso usando el token de refresco.

7.2.2 Usuarios y grupos

Método	Endpoint	Descripción
GET	/api/users/	Listar usuarios (admin: todos; usuario normal: solo él mismo).
POST	/api/users/	Crear usuario (solo admin).
PUT/PATCH	/api/users/{id}/	Actualizar usuario.
DELETE	/api/users/{id}/	Eliminar usuario (solo admin).
GET	/api/users-groups/	Listar grupos con sus permisos (solo admin).
POST	/api/users-groups/	Crear grupo (solo admin).
PUT/PATCH	/api/users-groups/{id}/	Actualizar grupo (solo admin).
DELETE	/api/users-groups/{id}/	Eliminar grupo (solo admin).
GET	/api/permissions/	Listar permisos disponibles (solo admin).

7.2.3 Dispositivos

Método	Endpoint	Descripción
GET	/api/devices/	Listar dispositivos (con filtro opcional <code>?classroom=ID</code>). Los no-admin solo ven los de sus aulas.
POST	/api/devices/{id}/shutdown/	Enviar orden de apagado al agente del dispositivo.
POST	/api/devices/{id}/toggle-internet/	Alternar el bloqueo de internet del dispositivo en el router Mikro-Tik.
PUT/PATCH	/api/devices/{id}/	Actualizar dispositivo (asignar aula, puerto, etc.).
DELETE	/api/devices/{id}/	Eliminar dispositivo (solo admin).

7.2.4 Aulas

Método	Endpoint	Descripción
GET	/api/classroom/	Listar aulas (admin: todas; profesor: solo las asignadas).
POST	/api/classroom/	Crear aula (solo admin).
PUT / PATCH	/api/classroom/{id}/	Actualizar aula (solo admin).
DELETE	/api/classroom/{id}/	Eliminar aula (solo admin).
POST	/api/classroom/{id}/toggle-global-internet/	Alternar el bloqueo de internet para todos los equipos del aula.

7.2.5 Equipos de red y hosts permitidos

Método	Endpoint	Descripción
GET	/api/network-device/	Listar equipos de red.
POST	/api/network-device/	Crear equipo de red (solo admin). Contraseña cifrada con Fernet.
PUT/PATCH	/api/network-device/{id}/	Actualizar equipo de red (solo admin).
DELETE	/api/network-device/{id}/	Eliminar equipo de red (solo admin).
GET	/api/allowed-hosts/	Listar hosts permitidos.
POST	/api/allowed-hosts/	Crear host permitido (solo admin).
PUT/PATCH	/api/allowed-hosts/{id}/	Actualizar host permitido (solo admin).
DELETE	/api/allowed-hosts/{id}/	Eliminar host permitido (solo admin).

7.3 Protocolo WebSocket

El sistema utiliza dos endpoints WebSocket:

- /ws/client/ — Conexión de los agentes (PCs del aula).
- /ws/dashboard/ — Conexión del frontend (navegador del profesor).

7.3.1 Mensajes del Agente → Servidor

Tipo	Descripción
startup	Reporte de encendido con información completa del sistema (MAC, IP, hostname, SO, CPU, RAM, disco).

heartbeat	Latido periódico con MAC e IP. Renueva la clave TTL en Redis.
shutdown_notice	Aviso de que el equipo se está apagando.
screenshot	Captura de pantalla en formato JPEG base64.
command_result	Resultado de un comando ejecutado.

7.3.2 Mensajes del Servidor → Agente

Tipo	Descripción
command	Orden enviada al agente (shutdown, request_screenshot, etc.).
ping	Comprobación de que el agente sigue respondiendo.

7.3.3 Mensajes del Servidor → Dashboard

Tipo (event)	Descripción
online	Un dispositivo se ha conectado.
offline	Un dispositivo se ha desconectado o su heartbeat ha expirado.
internet_blocked	Se ha bloqueado internet a un dispositivo.
internet_unblocked	Se ha desbloqueado internet a un dispositivo.
classroom_block	Se ha bloqueado internet a toda un aula.
classroom_unblock	Se ha desbloqueado internet a toda un aula.
screenshot	Captura de pantalla reenviada desde un agente.

7.4 Flujo de datos detallado

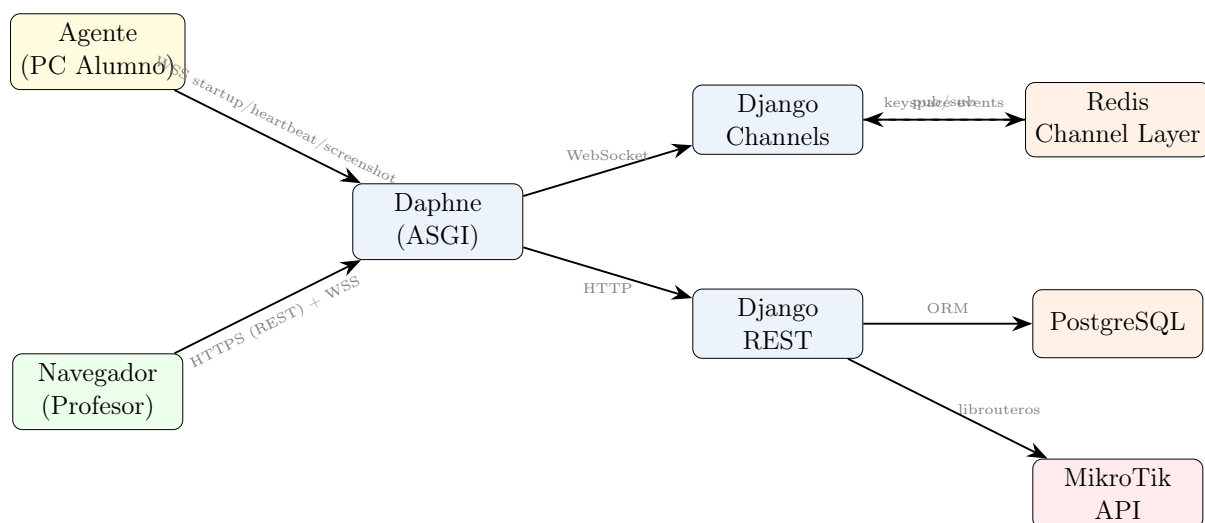


Figura 5: Flujo de datos entre los componentes del sistema.

7.5 Seguridad

7.5.1 Autenticación y autorización

- **JWT (JSON Web Tokens):** Autenticación sin estado. El token de acceso tiene una validez de 30 minutos y el de refresco 24 horas. El frontend los almacena en `localStorage` y los adjunta automáticamente con un interceptor de Axios.
- **RBAC con permisos granulares:** Se utiliza el sistema de permisos de Django extendido con `StrictDjangoModelPermissions`, que exige permisos incluso para peticiones GET (a diferencia del comportamiento por defecto de DRF).
- **Permiso `IsStaffForWrite`:** Las operaciones CRUD de escritura en recursos de infraestructura (aulas, equipos de red, hosts) requieren que el usuario sea `is_staff`. Sin embargo, las acciones personalizadas (`toggle-internet`, `shutdown`) están disponibles para cualquier usuario autenticado con permisos de visualización sobre el recurso.
- **`IsOwnerOrAdmin`:** Los usuarios no administradores solo pueden editar su propio perfil.

7.5.2 Cifrado y almacenamiento seguro

- **Contraseñas de equipos de red:** Se almacenan cifradas con Fernet (AES-128-CBC con HMAC) mediante un campo personalizado `EncryptedCharField`. La clave de cifrado se deriva de la `SECRET_KEY` del proyecto.
- **Contraseñas de usuarios:** Hash PBKDF2 con SHA-256 (mecanismo estándar de Django).
- **Comunicación TLS:** Todo el tráfico entre el navegador y el servidor pasa por HTTPS/WSS gracias a Caddy. La comunicación con la API de MikroTik también soporta TLS (puerto 8729).

7.5.3 CORS y CSRF

- En desarrollo, `CORS_ALLOW_ALL_ORIGINS = True` para facilitar las pruebas.
- En producción, los orígenes permitidos se configuran explícitamente en la variable de entorno `CORS_ALLOWED_ORIGINS`.
- Los orígenes de confianza CSRF se configuran automáticamente a partir de la misma variable.

8 Estructura del proyecto

```

1 proyecto_dam/
2 |-- backend/                                # Django 5 + DRF + Channels (Daphne)
3 |   |-- apps/
4 |   |   |-- devices/                        # Dispositivos, aulas, red, WebSocket
        consumers
5 |   |   |   |-- models.py                  # Device, Classroom, NetworkDevice,
        AllowedHost
6 |   |   |   |-- views.py                   # ViewSets con acciones
        toggle-internet, shutdown
7 |   |   |   |-- serializer.py
8 |   |   |   |-- consumers.py               # AgentConsumer, DashboardConsumer
9 |   |   |   |-- mikrotik_service.py        # Logica de interaccion con
        API MikroTik
10 |   |   |   |-- heartbeat_monitor.py      # Monitor Redis keyspace TTL
11 |   |   |   |-- fields.py                 # EncryptedCharField (Fernet)
12 |   |   |   |-- permissions.py
13 |   |   |   |-- routing.py                # Rutas WebSocket
14 |   |   |   +-- urls.py                   # Rutas REST
15 |   |   +-- users/                        # Usuarios, grupos, permisos, JWT
16 |   |       |-- models.py                 # User (extends AbstractUser)
17 |   |       |-- views.py                  # UserViewSet, GroupViewSet, JWT
18 |   |       |-- serializers.py
19 |   |       |-- permissions.py
20 |   |       +-- urls.py
21 |   |-- config/                           # Configuracion del proyecto
22 |   |   |-- settings.py                   # Configuracion Django
23 |   |   |-- urls.py                       # Rutas principales
24 |   |   +-- asgi.py                       # Punto de entrada ASGI
25 |   |-- Dockerfile.prod                   # Imagen Docker multi-stage
26 |   +-- requirements.txt
27 |-- frontend/                             # React 19 + Vite + Material UI 7 +
        Tauri 2
28 |   |-- src/
29 |   |   |-- components/                   # Tarjetas, modales, rutas protegidas
30 |   |   |-- hooks/                       # useAuth, useDashboardSocket
31 |   |   |-- pages/                       # Dashboard, Classrooms, Users, etc.
32 |   |   |-- stores/                      # Nanostores (screenshots)
33 |   |   |-- styles/                      # CSS + tema claro/oscuro
34 |   |   |-- api.js                       # Axios con interceptores JWT
35 |   |   +-- App.jsx                      # Rutas de la aplicacion
36 |   |-- src-tauri/                       # Nucleo Rust de la app de escritorio
37 |   |   |-- tauri.conf.json              # Configuracion Tauri (ventana,
        bundle, etc.)
38 |   |   |-- Cargo.toml                   # Dependencias Rust (tauri 2, serde,
        log)
39 |   |   |-- src/                         #Codigo Rust del nucleo (main.rs,
        lib.rs)
40 |   |   |-- capabilities/                # Permisos Tauri 2

```

```

41 | | +-- icons/                # Iconos de la aplicacion (.png,
   | | .icns, .ico)
42 | | -- init-tauri.sh         # Script de inicializacion automatica
   | | de Tauri
43 | +-- package.json
44 | -- client/                # Agente Python para PCs de alumnos
45 | | -- client.py            # Clase DeviceClient (WebSocket)
46 | | -- system_info.py       # Recopilacion de info del sistema
47 | | -- config.py            # Carga de configuracion
48 | | -- install_service.py   # Instalador como servicio del SO
49 | | -- requirements.txt
50 | +-- build/                # Sistema de compilacion con
   | | PyInstaller
51 | | -- build.py              # Script de compilacion
   | | multiplataforma
52 | +-- netmanagement_installer.spec # Spec de PyInstaller
53 | -- produccion/            # Configuracion de despliegue
54 | | -- docker-compose.prod.yml
55 | | -- Caddyfile
56 | | -- deploy.sh / deploy.bat
57 | +-- README.md
58 | -- docs/                  # Documentacion
59 | | -- database-er.mmd      # Modelo ER (Mermaid)
60 | +-- memoria.tex           # Este documento

```

Listing 1: Estructura de directorios del proyecto.

9 Implementación

9.1 Backend — Aspectos destacados

9.1.1 Cifrado transparente de contraseñas de red

Se ha implementado un campo de modelo personalizado `EncryptedCharField` que cifra automáticamente el valor al guardarlo en la base de datos y lo descifra al leerlo, de forma completamente transparente para el resto de la aplicación:

```

1 class EncryptedCharField(models.CharField):
2     def get_prep_value(self, value):
3         if value is None or value == '':
4             return value
5         f = _get_fernet()
6         return f.encrypt(value.encode()).decode()
7
8     def from_db_value(self, value, expression, connection):
9         if value is None or value == '':
10            return value
11        f = _get_fernet()
12        try:

```

```
13         return f.decrypt(value.encode()).decode()
14     except InvalidToken:
15         return value
```

Listing 2: Campo cifrado con Fernet (fields.py).

9.1.2 Monitor de heartbeat con Redis keyspace

El sistema de detección de equipos offline se basa en las notificaciones *keyspace* de Redis. Cada agente renueva una clave con TTL de 2 minutos en cada heartbeat. Cuando la clave expira (el agente dejó de enviar heartbeats), Redis notifica la expiración y un hilo demonio marca el dispositivo como offline:

```
1 def set_heartbeat(mac: str):
2     r = _get_redis()
3     r.set(f"{HEARTBEAT_PREFIX}{mac}", "1", ex=HEARTBEAT_TTL)
4
5 def _handle_expired_key(mac: str):
6     device = Device.objects.get(mac=mac)
7     if not device.is_online:
8         return
9     device.is_online = False
10    device.save()
11    # Notificar al dashboard via WebSocket
12    channel_layer = get_channel_layer()
13    async_to_sync(channel_layer.group_send)(
14        "dashboard_updates", {...}
15    )
```

Listing 3: Fragmento del monitor de heartbeat.

9.1.3 Servicio MikroTik — Network Fencing

El módulo `mikrotik_service.py` encapsula toda la lógica de interacción con la API de MikroTik. La estrategia de bloqueo combina dos capas:

1. **Capa L2 (Split-Horizon):** Se modifica el `horizon` del puerto del bridge para impedir el forwarding entre puertos con el mismo valor de horizon.
2. **Capa L3 (IP Firewall Filter):** Se crean reglas `accept` para cada host permitido seguidas de reglas `drop` para todo el tráfico restante del dispositivo.

Todas las reglas creadas se etiquetan con un comentario identificativo (`MONITOR_BLOCK_{id}`, `MONITOR_CLASS_BLOCK_{id}`) para poder eliminarlas selectivamente al desbloquear.

9.2 Frontend — Aspectos destacados

9.2.1 Gestión de tokens JWT con interceptores

El módulo `api.js` configura una instancia de `Axios` con interceptores que:

1. Adjuntan automáticamente el token de acceso en la cabecera `Authorization` de cada petición.
2. Si una petición recibe un error 401, intentan refrescar el token automáticamente y reintentan la petición original.
3. Si el refresco falla, redirigen al usuario a la página de login.

9.2.2 Capturas de pantalla con Nanostores

Las capturas de pantalla recibidas por `WebSocket` se almacenan en un `nanostore` persistente en `sessionStorage`. Esto permite que al cambiar de aula y volver, las últimas capturas se muestren inmediatamente sin esperar a que el agente envíe una nueva.

9.2.3 Rutas protegidas y permisos

El componente `ProtectedRoute` verifica:

- Que el usuario está autenticado (tiene token válido).
- Que tiene el permiso requerido (mediante el hook `useAuth`).
- Si la ruta requiere `requireStaff`, que el usuario es `staff`.

9.3 Agente cliente — Aspectos destacados

9.3.1 Compilación como binario autocontenido (PyInstaller)

Para facilitar la distribución masiva del agente en los equipos del aula, se ha creado un sistema de compilación basado en **PyInstaller**. El directorio `client/build/` contiene:

- `build.py`: Script de compilación que detecta automáticamente la plataforma (Windows o Linux), invoca `PyInstaller` con el spec correspondiente y genera el binario en `build/dist/{platform}/`.
- `netmanagement_installer.spec`: Spec de `PyInstaller` que empaqueta `install_service.py` como punto de entrada e incluye como datos adicionales los ficheros `client.py`, `config.py`, `system_info.py` y `requirements.txt`, necesarios en tiempo de ejecución. El resultado es un ejecutable único `NetManagement-Installer (.exe` en Windows) que puede distribuirse mediante GPO o copiado manual sin necesidad de instalar Python en el equipo destino.

```
1 cmd = [  
2     "pyinstaller",  
3     str(SPEC_FILE),  
4     "--distpath", str(dist_out),  
5     "--workpath", str(work_out),
```

```
6     "--noconfirm",  
7     "--clean",  
8 ]  
9 subprocess.run(cmd, check=True, cwd=str(CLIENT_DIR))
```

Listing 4: Fragmento del script de compilación build.py.

9.3.2 Instalación como servicio del sistema

El script `install_service.py` automatiza la instalación del agente como:

- **Windows:** Tarea programada del Task Scheduler que se ejecuta al iniciar sesión.
- **Linux:** Servicio `systemd` que se inicia con el sistema.

El instalador crea automáticamente un entorno virtual, instala las dependencias y registra el servicio.

9.3.3 Reconexión automática

El agente implementa reconexión automática con backoff configurable. Si la conexión WebSocket se pierde, reintenta la conexión cada `RECONNECT_DELAY` segundos (configurable, por defecto 5) hasta alcanzar el máximo de reintentos (configurable, 0 = infinito).

9.4 Aplicación de escritorio — Tauri 2

Además de funcionar como SPA web, el frontend de NetManagement puede distribuirse como una **aplicación de escritorio nativa** gracias a **Tauri 2**. Tauri es un framework que combina un núcleo en Rust con el WebView nativo del sistema operativo para renderizar la interfaz web, obteniendo binarios mucho más ligeros que Electron (que incluye Chromium completo).

9.4.1 Estructura del proyecto Tauri

La configuración de Tauri se encuentra en el directorio `frontend/src-tauri/`:

- `tauri.conf.json`: Fichero principal de configuración. Define el nombre del producto (*Net Management*), el identificador de la aplicación (`com.netmanagement.app`), la URL de desarrollo, la ruta de distribución y los ajustes de la ventana principal.
- `Cargo.toml`: Manifiesto de Rust. Declara las dependencias del núcleo: `tauri` (v2.10.3), `tauri-plugin-log` para logging nativo, `serde` y `serde_json`.
- `src/`: Código Rust del núcleo. En la versión actual, el núcleo es mínimo: solo se registran los plugins y se lanza la aplicación, delegando toda la lógica de negocio al frontend React.
- `capabilities/`: Permisos y capacidades de la aplicación Tauri (modelo de seguridad basado en `capabilities` de Tauri 2).

- **icons/**: Iconos de la aplicación en todos los formatos requeridos (32×32, 128×128, 128×128@2x, .icns para macOS, .ico para Windows).
- **init-tauri.sh**: Script de inicialización automática que instala **@tauri-apps/cli**, ejecuta **tauri init** en modo no interactivo y parcha el fichero de configuración.

9.4.2 Configuración de la ventana

La ventana principal de la aplicación de escritorio se ha configurado con las siguientes características:

Parámetro	Valor	Justificación
Título	Net Management	Nombre del producto.
Ancho	800 px	Anchura mínima adecuada para el dashboard.
Alto	600 px	Altura mínima adecuada.
Redimensionable	No	Evita que el layout se rompa en resoluciones imprevistas.
Pantalla completa	No	Modo ventana estándar.
CSP	null (deshabilitado)	La app hace peticiones a un dominio externo configurable; una CSP estática rompería la funcionalidad.

Cuadro 21: Configuración de la ventana Tauri de NetManagement.

9.4.3 Flujo de compilación

El proceso para generar el instalador de escritorio es el siguiente:

1. **Instalar Rust** (**rustup**) y las dependencias de sistema de Tauri (WebKitGTK en Linux).
2. Desde **frontend/**, ejecutar: **bun run tauri build**.
3. Tauri compila el frontend con Vite (**bun run build**), después compila el núcleo Rust y enlaza el resultado con el WebView del sistema.
4. Los artefactos generados en **src-tauri/target/release/bundle/** incluyen:
 - **.deb** y **.AppImage** en Linux.
 - **.msi** y **.exe** (NSIS) en Windows.
 - **.dmg** en macOS.

10 Despliegue

10.1 Requisitos del entorno

Herramienta	Versión mínima	Uso
Docker	24+	Contenedores de servicios
Docker Compose	v2+	Orquestación
Bun (o Node.js 18+)	1.0+	Compilación del frontend
Git	2.x	Clonado del repositorio
Router MikroTik	RouterOS 7.x	Infraestructura de red (opcional)

Cuadro 22: Requisitos del entorno de despliegue.

10.2 Servicios Docker

El fichero `docker-compose.prod.yml` define los siguientes servicios:

Servicio	Imagen	Función
db	postgres:16-alpine	Base de datos PostgreSQL con volumen persistente y healthcheck.
redis	redis:7-alpine	Channel Layers y heartbeat. Persistencia RDB cada 60 segundos.
backend	Build desde <code>Dockerfile.prod</code>	Django + Daphne. Imagen multi-stage para producción.
caddy	caddy:2-alpine	Proxy inverso con TLS automático. Único servicio que expone puertos (80, 443).

10.3 Proceso de despliegue

El script `deploy.sh` automatiza todo el proceso:

1. Verifica la existencia del fichero `.env` con las variables de entorno.
2. Instala dependencias y compila el frontend con Bun (`bun install && bun run build`).
3. Construye las imágenes Docker y levanta los contenedores.
4. Ejecuta las migraciones de base de datos.
5. Recopila los ficheros estáticos de Django (`collectstatic`).
6. Crea el superusuario inicial si no existe.

10.4 Variables de entorno

Variable	Descripción
POSTGRES_DB	Nombre de la base de datos.
POSTGRES_USER	Usuario de PostgreSQL.
POSTGRES_PASSWORD	Contraseña de PostgreSQL.
SECRET_KEY	Clave secreta de Django (se genera con <code>generar_clave.py</code>).
DEBUG	Modo debug (False en producción).
ALLOWED_HOSTS	Hosts permitidos separados por coma.
CORS_ALLOWED_ORIGINS	Orígenes CORS permitidos.
DOMAIN	Dominio para Caddy (TLS automático).
DJANGO_SUPERUSER_*	Credenciales del superusuario inicial.

11 Pruebas

11.1 Pruebas funcionales realizadas

Se han realizado pruebas manuales exhaustivas de todas las funcionalidades del sistema:

ID	Funcionalidad	Descripción de la prueba	Resultado
T-01	Registro automático	Un agente se conecta y aparece en el dashboard automáticamente.	OK
T-02	Detección offline	Se cierra el agente y el dispositivo pasa a offline tras expirar el heartbeat.	OK
T-03	Apagado remoto	Se envía la orden de apagado y el equipo se apaga correctamente.	OK
T-04	Bloqueo individual	Se bloquea internet a un equipo y se verifica que pierde conectividad.	OK
T-05	Desbloqueo individual	Se desbloquea internet y se verifica la restauración de conectividad.	OK
T-06	Bloqueo por aula	Se bloquea internet a toda un aula y se verifica en todos los equipos.	OK
T-07	Hosts permitidos	Durante un bloqueo, los hosts permitidos siguen siendo accesibles.	OK
T-08	Capturas de pantalla	Se reciben capturas periódicas en el dashboard.	OK
T-09	RBAC	Un profesor solo ve las aulas asignadas y no puede acceder a configuración.	OK
T-10	Refresco JWT	El token se refresca automáticamente sin interrumpir la sesión.	OK
T-11	Reconexión agente	El agente reconecta automáticamente tras una caída del servidor.	OK

T-12	Auto- descubrimiento puerto	Al conectarse un equipo, se detecta su puerto del switch automáticamente.	OK
------	-----------------------------------	--	----

11.2 Entornos de prueba

- **Desarrollo local:** Backend con `runserver`/`Daphne`, PostgreSQL y Redis en Docker, frontend con `vite dev`.
- **Producción:** Docker Compose completo con Caddy, desplegado en servidor con acceso a router MikroTik real.
- **Equipos cliente:** Probado en Windows 10/11, Ubuntu 24.04 y Arch Linux con escritorio.

12 Complicaciones encontradas

A lo largo del desarrollo del proyecto se encontraron diversas dificultades técnicas que requirieron investigación adicional y, en algunos casos, soluciones alternativas o limitaciones aceptadas.

12.1 Instalación del agente como servicio en Windows

Uno de los objetivos del agente cliente era poder ejecutarse como un servicio del sistema operativo, de forma que se iniciase automáticamente al arrancar el equipo sin necesidad de que el usuario interactuase.

En **Linux** esto se resolvió de forma sencilla mediante un fichero de unidad `systemd` que ejecuta el intérprete de Python del entorno virtual con el script del agente.

En **Windows**, sin embargo, la situación fue considerablemente más compleja. Se intentaron varias aproximaciones:

- **Tarea programada del Task Scheduler:** Fue la primera opción, creando una tarea que se ejecutase al iniciar sesión (ONLOGON). El problema principal fue que el Task Scheduler tiene dificultades para localizar y ejecutar correctamente el intérprete de Python dentro de un entorno virtual, especialmente cuando la ruta contiene espacios o caracteres especiales. Los permisos de ejecución en el contexto del usuario local también generaron problemas intermitentes.
- **NSSM (Non-Sucking Service Manager):** Se evaluó el uso de `nssm`, una herramienta de terceros que permite envolver cualquier ejecutable como un servicio nativo de Windows. Aunque conceptualmente era la solución ideal, la configuración programática desde Python resultó problemática: la detección del entorno virtual, la propagación de variables de entorno y la gestión de la salida estándar del proceso no funcionaban de forma fiable en todos los entornos probados.

Finalmente, se optó por mantener la instalación como **tarea programada del Task Scheduler** como la opción más compatible, aunque con limitaciones conocidas en cuanto a la fiabilidad de inicio automático en todas las configuraciones de Windows.

12.2 Capturas de pantalla en Linux con Wayland

La librería **mss** (python-mss) se utiliza para la captura de pantalla en el agente. Esta librería funciona accediendo directamente al servidor gráfico del sistema operativo para obtener una imagen del escritorio.

En **Linux con X11/Xorg** (el servidor gráfico tradicional), **mss** funciona correctamente, ya que X11 permite a cualquier proceso conectado al servidor X acceder a los píxeles del framebuffer de la pantalla sin restricciones significativas.

Sin embargo, en **Linux con Wayland** (el protocolo de servidor gráfico más moderno que está sustituyendo a X11 en distribuciones recientes como Ubuntu 22.04+, Fedora o GNOME en Arch), la captura de pantalla **no funciona**. Esto se debe a que Wayland implementa un modelo de seguridad fundamentalmente diferente:

- En **X11**, cualquier cliente X puede leer los píxeles de cualquier ventana. No existe aislamiento entre aplicaciones gráficas. Esto facilita las capturas, pero es un problema de seguridad.
- En **Wayland**, cada cliente solo tiene acceso a sus propios buffers. Para capturar la pantalla completa, se requiere utilizar portales de escritorio (**xdg-desktop-portal**) y la API de PipeWire/ScreenCast, que exigen interacción del usuario (un diálogo de consentimiento) la primera vez.

Dado que **mss** accede directamente al protocolo X11, no es compatible con Wayland de forma nativa. La solución adoptada fue **documentar la limitación** e indicar que los equipos del aula donde se instale el agente deben utilizar **X11/Xorg** como servidor gráfico (en la mayoría de distribuciones se puede seleccionar en la pantalla de inicio de sesión). En entornos educativos, donde los equipos suelen tener configuraciones controladas por el administrador, esto es una restricción asumible.

12.3 Certificados TLS con Caddy y conexiones WSS del agente

Caddy 2 gestiona los certificados TLS de forma automática mediante Let's Encrypt. En un entorno de producción con un dominio público, esto funciona sin intervención manual. Sin embargo, durante el desarrollo y las pruebas en **red local** (sin dominio público), surgieron dificultades:

- **Certificados autofirmados:** Caddy genera certificados autofirmados cuando no puede obtener un certificado de Let's Encrypt. Los agentes Python, al conectarse por **wss://** (WebSocket sobre TLS), rechazaban estos certificados al no estar firmados por una Autoridad Certificadora reconocida.
- **Solución en desarrollo:** Se configuró el agente con la opción **ssl=False** en las pruebas locales, deshabilitando la verificación TLS. Para pruebas con TLS se exportó

el certificado raíz de Caddy y se configuró como CA de confianza en los equipos cliente.

- **Solución en producción:** Con un dominio público y puertos 80/443 accesibles, Caddy obtiene certificados válidos de Let's Encrypt automáticamente y los agentes se conectan sin problemas.

Esta complicación puso de manifiesto la importancia de planificar la infraestructura de certificados desde las fases iniciales del proyecto.

12.4 Conexión con la API de MikroTik

La conexión con la API de MikroTik RouterOS mediante `librouteros` presentó varios problemas durante la implementación:

- **Puertos y protocolos:** MikroTik ofrece dos puertos de API: el 8728 (sin TLS) y el 8729 (con TLS/SSL). La conexión TLS requiere configuración adicional tanto en el router (certificado SSL) como en el cliente Python (verificación o deshabilitación de la verificación del certificado). Durante el desarrollo, la conexión sin TLS (puerto 8728) fue más práctica, mientras que en producción se recomienda utilizar el puerto con TLS.
- **Habilitación de la API:** Por defecto, la API de MikroTik no está habilitada. Es necesario activarla explícitamente en `/ip/service` del router y configurar las direcciones permitidas que pueden conectarse.
- **Formato de las consultas:** La API de MikroTik utiliza un protocolo binario propio con una sintaxis de consulta particular (filtros con `?`, modificadores con `=`) que difiere significativamente de las APIs REST habituales. La documentación de `librouteros` fue esencial para comprender la traducción entre las rutas del CLI de RouterOS y las llamadas de la API.
- **Timeout y reconexión:** Las conexiones a la API pueden cerrarse por inactividad. Se implementó la apertura y cierre de conexión por cada operación en lugar de mantener una conexión persistente, evitando así problemas de timeout.

13 Conclusiones

13.1 Objetivos alcanzados

Se han cumplido todos los objetivos definidos para el MVP:

- Se ha desarrollado un sistema completo de monitorización y control de equipos en aulas informáticas con interfaz web.
- Se ha implementado la funcionalidad diferenciadora del proyecto: el *Network Fencing* mediante integración directa con la API de MikroTik, permitiendo el aislamiento de red a nivel físico (L2/L3).

- Se ha diseñado e implementado un agente multiplataforma ligero que se instala como servicio del sistema.
- Se ha implementado un sistema de comunicación en tiempo real con WebSockets para monitorización instantánea.
- Se ha proporcionado un despliegue automatizado con Docker Compose y TLS automático.

13.2 Lecciones aprendidas

- La combinación de split-horizon (L2) con reglas de IP firewall (L3) fue la solución más robusta para el aislamiento de red, ya que bloquea tanto el tráfico bridgeado como el enrutado.
- Django Channels con Redis como backend resulta una solución fiable para WebSockets en tiempo real, aunque requiere una configuración cuidadosa de la capacidad del *channel layer* para mensajes grandes (capturas de pantalla).
- El sistema de heartbeat con Redis keyspace notifications es elegante y eficiente, pero depende de que Redis esté configurado con `notify-keyspace-events` habilitado.
- La instalación del agente como servicio del sistema en diferentes SO requiere un manejo cuidadoso de las particularidades de cada plataforma (Task Scheduler en Windows, systemd en Linux).

14 Líneas de trabajo futuro

A continuación se describen las principales líneas de mejora identificadas para evolucionar el sistema hacia un estándar de calidad corporativo, superando las limitaciones propias de un prototipo académico.

- **Seguridad en WebSockets:** Actualmente, la conexión WebSocket del agente se establece sin un mecanismo de autenticación mutua previo al intercambio de datos. Una mejora prioritaria consiste en implementar un protocolo de autenticación inicial —por ejemplo, mediante tokens JWT de corta duración o secretos compartidos (*pre-shared keys*)— que obligue al agente a demostrar su identidad antes de que el servidor acepte la conexión. De este modo se mitigaría el riesgo de inyección de telemetría falsa por parte de clientes no autorizados.
- **Validación estricta de certificados SSL/TLS:** Durante la fase de desarrollo, la verificación del certificado del servidor se deshabilitó para facilitar las pruebas en red local con certificados autofirmados. En un entorno de producción real, resulta imprescindible configurar certificados emitidos por una Autoridad Certificadora reconocida y habilitar la validación estricta tanto en el agente como en las conexiones a la API de MikroTik, eliminando así la superficie de ataque frente a ataques de tipo *Man-in-the-Middle* (MitM).
- **Automatización mediante CI/CD:** La incorporación de un *pipeline* de integración y entrega continuas —por ejemplo, mediante GitHub Actions— permitiría

ejecutar de forma automática las suites de pruebas unitarias, los *linters* de código y los análisis estáticos con cada *push* o *pull request*. Esto garantizaría la detección temprana de regresiones y el mantenimiento de un nivel de calidad de código homogéneo a lo largo de todo el ciclo de vida del proyecto.

- **Empaquetado y despliegue masivo del agente:** Actualmente, el agente se distribuye como un script Python que requiere un intérprete y dependencias instaladas en cada equipo. Una evolución natural consiste en compilar el agente como un ejecutable binario independiente mediante herramientas como PyInstaller y encapsularlo en un instalador silencioso (`.msi`). Esto posibilitaría el despliegue remoto y transparente en todas las aulas del centro educativo utilizando Directivas de Grupo (GPO) de Windows Server, sin intervención manual en cada equipo.
- **Refinamiento en sistemas UNIX:** En equipos con macOS, la ejecución de determinadas órdenes remotas —como el apagado del sistema— requiere privilegios de superusuario. Actualmente, esta operación puede exigir interacción manual para la introducción de credenciales. Una línea de mejora consiste en configurar de forma automatizada el archivo `sudoers` (mediante `visudo`) para conceder al usuario del agente permisos específicos sobre los comandos necesarios, permitiendo así una gestión remota completamente desatendida.

15 Bibliografía y referencias

1. Django Software Foundation. *Django documentation (v5.2)*. <https://docs.djangoproject.com/en/5.2/>
2. Django REST Framework. *DRF documentation*. <https://www.django-rest-framework.org/>
3. Django Channels. *Channels documentation*. <https://channels.readthedocs.io/>
4. Meta Platforms, Inc. *React documentation (v19)*. <https://react.dev/>
5. Material UI. *MUI documentation (v7)*. <https://mui.com/>
6. Vite. *Vite documentation*. <https://vitejs.dev/>
7. Redis Ltd. *Redis documentation*. <https://redis.io/docs/>
8. PostgreSQL Global Development Group. *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
9. MikroTik. *RouterOS documentation*. <https://help.mikrotik.com/docs/>
10. librouteros. *Python MikroTik API client*. <https://librouteros.readthedocs.io/>
11. Caddy Web Server. *Caddy documentation*. <https://caddyserver.com/docs/>
12. Docker, Inc. *Docker Compose documentation*. <https://docs.docker.com/compose/>
13. Nanostores. *Nanostores — tiny state manager*. <https://github.com/nanostores/nanostores>
14. websockets. *websockets Python library*. <https://websockets.readthedocs.io/>

15. psutil. *psutil* — cross-platform process and system utilities. <https://psutil.readthedocs.io/>
16. mss. *python-mss* — fast screenshots. <https://python-mss.readthedocs.io/>
17. Tauri. *Tauri v2 documentation*. <https://v2.tauri.app/>
18. The Rust Programming Language. *The Rust Reference*. <https://doc.rust-lang.org/reference/>
19. PyInstaller. *PyInstaller documentation*. <https://pyinstaller.org/en/stable/>